

EECS 2311 – Lab 5: Take-Home Assignment

QA, Refactoring, and Optimization

Field	Value
Name & ID	Nurjahan Ahmed Shiah - 218802348
Course	EECS 2311 - Z
Team	Group 02
Project	Fitiva - Fitness App

Activity 1: Tester Assignment

1.1 Fully Implemented User Stories with GUI

Big Story 1: Create a Fitness Profile & Personalized Workout

- User Story 1.1: Register & Log In with Email (ITR1) - As a new user/Trainer, I want to create an account using my email and password so that I can securely save my profile, workout plans, and progress. As a returning user, I want to log in so that I can access my existing plans and workout history.
 - Ege Yesilyurt
- User Story 1.2: Create Fitness Profile (ITR1) - As a new user, I want to enter my age, experience level, training location, and primary fitness focus so that the system can recommend suitable workout plans. As a trainer, I want to create a professional profile and publish my programs so that users can discover my content.
 - Raha Golsorkhi: implemented the backend for filling out profile features such as name, age, level of expertise.
 - Arshia Hassanpour
- User Story 1.3: Create Programs from the list of workouts (ITR1) - As a trainer, I want to create workout programs from a list of available exercises so that I can design structured training plans for my clients.
 - Weiqin Situ
- User Story 1.4: View List of Workouts (ITR1) - As a user, I want to view a list of available workouts so that I can explore individual sessions and understand what types of training are offered before committing to a plan or program.
 - Ege Yesilyurt
- User Story 1.5: Browse Trainer Created Programs (ITR1) - As a user, I want to browse workout programs created by trainers so that I can explore different training styles and subscription-based options.
 - Nurjahan Shiah
- User Story 1.6: View profile-based workout recommendations (ITR1) - As a user, I want to see workout plans recommended based on my fitness profile so that I can quickly find programs that match my goals and experience level.
 - Ege Yesilyurt

- User Story 1.7: Select training plan and auto-generate weekly schedule (ITR1) - As a user, I want to select a workout plan and have the system automatically generate my weekly training schedule so that I know what workouts to perform each day.
 - Nurjahan Shiah

Big Story 2: Reflect & Adapt Training Plans

- User Story 2.1: Submit Post-Workout Feedback & Trainer Management (ITR2) - As a user, I want to rate workout difficulty and provide session feedback so that the system can personalize and improve my future training plans.
 - Ege Yesilyurt (main): Implemented the post-workout feedback forms and backend logic.
 - Weiqin Situ (sub-task): Implemented the trainer-side Program Details edit page (/edit-program/[id]), allowing trainers to edit program details (excluding name). Also added a soft-delete program button with a confirmation pop-up, linked to the is_deleted flag to keep track of dashboard stats.
- User Story 2.2: View Plan Adjustments & Explanations (ITR3) - As a user, I want to see which parts of my next week's training plan were changed based on my workout feedback so that I understand why those adjustments were made.
 - Dawood
- User Story 2.3: Automatic Weekly Schedule Regeneration (ITR2) - As a user, I want to rate how difficult a workout felt and report issues so that the system can learn from my experience.
 - Nurjahan Shiah
- User Story 2.4: Review Aggregated Client Feedback (ITR2) - As a trainer, I want to review aggregated feedback from users following my programs so that I can evaluate program effectiveness and make improvements.
 - Weiqin Situ
- User Story 2.5: Accept or Lock Recommended Adjustments (ITR2) - As a user, I want to accept, reject, or lock recommended schedule changes so that I retain control over my training plan.
 - Nurjahan Shiah

Big Story 3: Performance Analysis & Progress Dashboard

- User Story 3.1: Record Workout Completion (ITR1) - As a user, I want to mark a workout as completed and log basic details so that my training history and statistics remain accurate.
 - Raha Golsorkhi

- User Story 3.2: Review Workout History (ITR3) - As a user, I want to browse my completed workouts in chronological order so that I can review past activity and maintain accountability.
 - Raha Golsorkhi
- User Story 3.3: Analyze Training Trends (ITR2) - As a user, I want to view summarized insights from my completed workouts so that I can understand my training patterns and monitor my consistency over time.
 - Arshia Hassanpour
- User Story 3.4: View Progress Summary Dashboard (ITR2) - As a user, I want to see a concise dashboard that summarizes my key performance metrics so that I can quickly understand my overall progress.
 - Raha Golsorkhi
- User Story 3.5: Detect Missed Sessions (ITR3) - As a user, I want missed workouts to be reflected in my activity records so that my consistency metrics remain accurate.
 - Arshia Hassanpour
- User Story 3.6: Personalized Schedule from Selected Program (ITR1) - As a user, I want to generate a personalized weekly workout schedule based on the program I selected, so that I can follow a structured and organized plan tailored to my goals and availability.
 - Nurjahan Shiah

Big Story 4: Engagement & Reward Mechanisms

- User Story 4.1: Earn Points for Workout Completion (ITR3) - As a user, I want to earn points when I complete workouts so that my effort is consistently recognized.
 - Dawood
- User Story 4.2: Unlock Achievement Badges (ITR3) - As a user, I want to unlock achievement badges when I reach milestones so that my progress feels rewarding and visible.
 - Dawood
 - Arshia
- User Story 4.3: View Achievement Gallery (ITR3) - As a user, I want to browse a gallery of my earned badges and rewards so that I can track accomplishments over time.
 - Arshia Hassanpour
- User Story 4.4: Participate in Weekly Challenges (ITR3) - As a user, I want to join time-limited workout challenges so that I have short-term goals to keep me motivated.
 - Weiqin Situ

- User Story 4.5: Trainer-Hosted Challenges (ITR3) - As a trainer, I want to create themed workout challenges so that I can engage users and promote my programs.
 - Ege Yesilyurt

The following user stories that I am responsible for testing:

Story ID	User Story Title	Parent Epic	Estimate	Priority	Main Developer
US 2.1 (Shared testing with Dawood Al-Janaby)	Submit Post-Workout Feedback	Big Story 2: Reflect and Adapt	3 days	High	Ege Yesilyurt
US 3.1	Record Workout Completion	Big Story 3: Performance Tracking	3 days	High	Raha Golsorkhi
US 3.2	Review Workout History	Big Story 3: Performance Tracking	3 days	High	Raha Golsorkhi
US 4.4	Participate in Weekly Challenges	Big Story 4: Gamification	2 days	Low	Wei Qin Situ

1.2 Main Developers per User Story (Detailed Tasks)

US 2.1: Submit Post-Workout Feedback

- Backend (feedback endpoint, WorkoutFeedback model, difficulty_rating/ fatigue_level/ pain_reported fields): Ege Yesilyurt
- Frontend (feedback form on schedule page, API calls to workout_feedback endpoint): Ege Yesilyurt
- Integration with adaptive scheduling (feedback feeding into analyze_feedback): Ege Yesilyurt

US 3.1: Record Workout Completion

- Backend (complete_workout_session view, session status logic, duration auto-detection): Raha Golsorkhi
- Frontend (schedule page workout cards, Start/Complete buttons, session state): Raha Golsorkhi
- Points and badge awarding on completion (award_points, check_and_award_badges): Raha Golsorkhi

US 3.2: Review Workout History

- Backend (workout_history view, date-range filtering, auto-marking missed sessions): Raha Golsorkhi
- Frontend (history page, date pickers, session cards with status/plan name/duration): Raha Golsorkhi

US 4.4: Participate in Weekly Challenges

- Backend (Challenge & UserChallenge models, join_challenge, challenge progress update in complete_workout_session): Weiqin Situ
- Frontend (challenges page, tab UI for global/trainer challenges, join flow): Weiqin Situ
- Reward logic (points/badge awarded when challenge completed): Weiqin Situ

1.3 Testing Assignment Distribution

Tester: Weiqin Situ

- US 1.2 (Create Fitness Profile)
- US 3.5 (Detect Missed Sessions)
- US 3.6 (Personalized Schedule from Program)
- US 4.1 (Earn Points for Workout Completion)

Tester: Ege Yesilyurt

- US 1.3 (Create Programs from workouts)
- US 2.3 (Automatic Weekly Schedule Regeneration)
- US 3.3 (Analyze Training Trends)
- US 4.2 (Unlock Achievement Badges)

Tester: Arshia Hassanpour

- US 1.4 (View List of Workouts)
- US 2.4 (Review Aggregated Client Feedback)
- US 4.5 (Trainer-Hosted Challenges)
- US 2.2 (View Plan Adjustments & Explanations)

Tester: Raha Golsorkhi

- US 1.5 (Browse Trainer Created Programs)
- US 2.5 (Accept or Lock Recommended Adjustments)
- US 1.6 (View profile-based recommendations)
- US 4.3 (View Achievement Gallery)

Tester: Dawood Al-Janaby

- US 1.7 (Select plan & auto-generate schedule)

- US 3.4 (View Progress Summary Dashboard)
- US 1.1 (Register & Log In with Email)
- US 2.1 (Submit Post-Workout Feedback & Trainer Management) - *shared testing with Shiah*

Tester: Nurjahan Ahmed Shiah

- US 4.4 (Participate in Weekly Challenges)
- US 3.2 (Review Workout History)
- US 3.1 (Record Workout Completion)
- US 2.1 (Submit Post-Workout Feedback & Trainer Management) - *shared testing with Dawood*

Activity 2: Individual Acceptance Testing

US 2.1: Submit Post-Workout Feedback

User Story: *As a user, I want to rate how a workout felt and provide quick feedback so that the system can adapt my future training plans to my real performance and energy levels.*

TC-2.1-01: Submit Feedback After Completing a Workout

Test ID	TC-2.1-01
Title	User submits difficulty rating and optional fatigue level after completing a session
Pre-conditions	User is logged in. Today has a completed WorkoutSession (status='completed').
Steps	1. Complete a workout session (or use a pre-completed session). 2. Navigate to the feedback form (triggered after completion or via schedule page). 3. Select a difficulty rating of 4 (Hard) on the 1-5 scale. 4. Optionally select fatigue level 3. 5. Leave pain_reported unchecked. 6. Submit feedback (click confirm/save). 7. Verify a success message appears. 8. In DB, verify WorkoutFeedback record with difficulty_rating=4, fatigue_level=3, pain_reported=False exists linked to this session.
Expected Result	WorkoutFeedback saved to DB. Feedback completes in under 30 seconds. Success notification shown. Schedule page updates to show feedback has been submitted.
Status	PASS / FAIL

TC-2.1-02: Skip Feedback

Test ID	TC-2.1-02
Title	User skips the feedback form without submitting
Pre-conditions	User is logged in. Today has a completed WorkoutSession.
Steps	1. Complete a workout session. 2. When the feedback prompt appears, click 'Skip' or dismiss the modal. 3. Verify the app returns to the schedule page without any error. 4. In DB, verify no WorkoutFeedback record was created for this session.
Expected Result	Feedback is optional. Dismissing the prompt does not create a DB record or block the user from continuing. Session remains completed.
Status	PASS / FAIL

TC-2.1-03: Submit Feedback with Pain Reported

Test ID	TC-2.1-03
Title	User reports pain during feedback, triggering adaptive schedule suggestions
Pre-conditions	User is logged in. Today has a completed WorkoutSession. User has an active schedule.
Steps	1. Complete a workout session. 2. Open the feedback form. 3. Select difficulty rating 5 (Very Hard). 4. Check 'pain_reported'. 5. Submit feedback. 6. Navigate to the schedule page. 7. Verify an AI adjustment suggestion appears mentioning pain recovery options. 8. In DB: WorkoutFeedback.pain_reported=True for this session.
Expected Result	Feedback saved with pain_reported=True. Schedule page shows a pain-recovery adjustment suggestion with recovery options (rest, lighter focus, etc.). No crash.
Status	PASS / FAIL

TC-2.1-04: Cannot Submit Feedback for Incomplete Session (Error Path)

Test ID	TC-2.1-04
Title	Submitting feedback for a session that is not completed returns an error
Pre-conditions	User is logged in. Today has a WorkoutSession with status='in_progress' (not completed).
Steps	1. Log in and start (but do not complete) a workout session. 2. Directly POST to /api/workout/2026-03-28/feedback/ with difficulty_rating=3. 3. Observe the API response.
Expected Result	API returns HTTP 400 with message 'Cannot submit feedback for an incomplete workout session'. No WorkoutFeedback record is created in DB.
Status	PASS / FAIL

TC-2.1-05: Feedback Difficulty Rating Validation

Test ID	TC-2.1-05
Title	Submitting an out-of-range difficulty rating returns a validation error
Pre-conditions	User is logged in. Today has a completed WorkoutSession.
Steps	1. Complete a workout session. 2. Submit a POST to /api/workout/[today's date]/feedback/ with difficulty_rating=6 (out of 1-5 range). 3. Observe the API response.
Expected Result	API returns HTTP 400 with message 'difficulty_rating must be an integer between 1 and 5'. No feedback record is created in DB.
Status	PASS / FAIL

US 3.1: Record Workout Completion

User Story: *As a user, I want to mark a workout as completed and log basic details so that my training history and statistics remain accurate.*

TC-3.1-01: View Today's Scheduled Workout and Start Session

Test ID	TC-3.1-01
Title	User views today's workout plan with exercises and starts a session
Pre-conditions	User is logged in. User has an active schedule assigned. Today is a workout day in the weekly_schedule.
Steps	1. Log in and navigate to the Schedule page. 2. Verify today's date card shows the workout plan name and exercises (with sets, reps, rest time). 3. Click 'Start Workout'. 4. Verify the button changes state (e.g., session is now 'in_progress'). 5. In the DB, verify a WorkoutSession with status='in_progress' exists for today.
Expected Result	WorkoutSession created with status='in_progress'. UI reflects the started state. Exercise details (sets, reps, rest) are visible.
Status	PASS / FAIL

TC-3.1-02: Complete a Workout and Verify Stats

Test ID	TC-3.1-02
Title	User completes a workout and the session is saved with correct duration
Pre-conditions	User is logged in. Today has a started ('in_progress') WorkoutSession.
Steps	1. Start a workout session (or use TC-3.1-01 result). 2. Click 'Complete Workout'. 3. Verify a success notification appears showing points awarded. 4. Navigate to Workout History; confirm today's session appears with status 'completed' and correct plan name. 5. In the DB, verify WorkoutSession.is_completed=True, status='completed', and duration_minutes is auto-populated from the program's session_length.
Expected Result	WorkoutSession marked completed. Duration auto-filled. Session visible in history. Points awarded. DB record is consistent.
Status	PASS / FAIL

TC-3.1-03: No Workout Scheduled: Empty State

Test ID	TC-3.1-03
Title	User has no workout scheduled for today and sees appropriate messaging
Pre-conditions	User is logged in. Today is a rest day or user has no active schedule.
Steps	1. Log in on a day that is a rest day in the schedule. 2. Navigate to the Schedule page. 3. Observe the today card. 4. Verify no 'Start Workout' button is shown for a rest day. 5. Verify a rest day indicator or message is displayed.
Expected Result	Rest day is clearly indicated. No workout actions are available. No session is created in the DB.
Status	PASS / FAIL

TC-3.1-04: Undo Workout Completion

Test ID	TC-3.1-04
Title	User undoes a completed workout session, reverting it to in-progress
Pre-conditions	User is logged in. Today has a completed WorkoutSession.
Steps	1. Complete a workout session. 2. If an 'Undo' option is available in the UI, click it. 3. Verify the session reverts to 'in_progress'. 4. Verify associated WorkoutFeedback is deleted. 5. In DB, confirm WorkoutSession.is_completed=False and status='in_progress'.
Expected Result	Session is reset. Feedback is removed. 'Complete Workout' button re-enables. DB consistent.
Status	PASS / FAIL

TC-3.1-05: Completing Session Awards Points and Badge

Test ID	TC-3.1-05
Title	Completing a workout awards the correct points and triggers badge check
Pre-conditions	User is logged in. User has fewer sessions than any badge threshold.

Steps	1. Log in and complete today's workout session. 2. Observe the completion response: check for 'points_awarded' value. 3. Navigate to Rewards/Profile page. 4. Verify total points increased by the displayed amount. 5. In DB, verify PointTransaction record exists for this session. Verify UserPoints.total_points updated.
Expected Result	Points awarded match the displayed amount. PointTransaction created. UserPoints updated. No duplicate transactions.
Status	PASS / FAIL

US 3.2: Review Workout History

***User Story:** As a user, I want to browse my completed workouts in chronological order so that I can review past activity and maintain accountability.*

TC-3.2-01: View Full Workout History

Test ID	TC-3.2-01
Title	User views their complete workout history without any filters
Pre-conditions	User is logged in. User has at least 2 completed or missed WorkoutSession records in the DB.
Steps	1. Log in and navigate to Workout History (from dashboard or nav). 2. Verify the page title 'Workout History' is displayed. 3. Confirm session cards are listed, each showing: a formatted date, plan name (or 'Missed workout'), and duration or '--'. 4. Verify sessions are ordered from most recent to oldest (descending date order).
Expected Result	All completed and missed sessions are visible. Order is newest-first. Session cards display correct date, plan name, and duration.
Status	PASS / FAIL

TC-3.2-02: Filter History by Date Range

Test ID	TC-3.2-02
Title	User filters workout history using start and end date filters
Pre-conditions	User is logged in. User has sessions both inside and outside a known date range (e.g., 2026-01-01 to 2026-01-31).

Steps	1. Log in and navigate to Workout History. 2. Enter '2026-01-01' in the Start Date input. 3. Enter '2026-01-31' in the End Date input. 4. Click 'Apply Filter'. 5. Verify only sessions within Jan 2026 are displayed. 6. Click 'Reset'; verify all sessions return and the date inputs are cleared.
Expected Result	Only sessions matching the date range are shown after filtering. Reset correctly clears filters and reloads all history.
Status	PASS / FAIL

TC-3.2-03: Missed Workouts Shown in History

Test ID	TC-3.2-03
Title	Missed sessions appear in history with correct label
Pre-conditions	User is logged in. A WorkoutSession with status='missed' exists in the DB for this user.
Steps	1. Log in and navigate to Workout History. 2. Locate the session card for the missed date. 3. Verify the plan name column shows 'Missed workout'. 4. Verify the duration column shows 'Missed'.
Expected Result	Missed sessions are visually distinguished from completed sessions with the 'Missed workout' and 'Missed' labels.
Status	PASS / FAIL

TC-3.2-04: Empty History State

Test ID	TC-3.2-04
Title	New user with no sessions sees an empty state with a link to the schedule
Pre-conditions	User is newly registered with no WorkoutSession records in the DB.
Steps	1. Log in as a new user. 2. Navigate to Workout History. 3. Verify the message 'No activity records yet.' is displayed. 4. Verify a 'Go to Schedule' link is present and navigates to /schedule.
Expected Result	Empty-state text visible. No session cards rendered. Schedule link is functional.

Status	PASS / FAIL
--------	-------------

TC-3.2-05: Invalid Date Range (Edge Case)

Test ID	TC-3.2-05
Title	User applies an end date before the start date
Pre-conditions	User is logged in and has session history.
Steps	1. Log in and navigate to Workout History. 2. Enter '2026-03-01' as Start Date. 3. Enter '2026-01-01' as End Date (before start). 4. Click 'Apply Filter'. 5. Observe the results.
Expected Result	Either: no sessions are returned (backend respects logical date ordering), or the UI displays a validation warning. No crash or unhandled exception occurs.
Status	PASS / FAIL
Bug Reported	Bug #1 if no validation: end date before start date returns results silently — should display an error.

US 4.4: Participate in Weekly Challenges

User Story: *As a user, I want to join time-limited workout challenges so that I have short-term goals to keep me motivated.*

TC-4.4-01: Browse and Join a Global Challenge

Test ID	TC-4.4-01
Title	User browses available global challenges and successfully joins one
Pre-conditions	User is logged in. At least one active global challenge exists in the DB with start_date <= today <= end_date.
Steps	1. Navigate to the app and log in with valid credentials. 2. From the dashboard or navigation, click 'Challenges'. 3. Verify the Challenges page loads showing the 'Global Challenges' tab by default. 4. Confirm at least one challenge card is visible, showing name, date range, goal criteria badges, and reward info. 5. Click 'Join Challenge' on one of the listed challenges. 6. Verify the button shows 'Joining...' briefly, then the page redirects to the dashboard.

	7. Navigate back to Challenges page; confirm the challenge no longer shows a 'Join' button (or shows joined state).
Expected Result	Challenge is joined. A UserChallenge record is created in the DB for this user+challenge combination. Dashboard reflects the active challenge. No error messages appear.
Status	PASS / FAIL

TC-4.4-02: Browse Trainer Challenges Tab

Test ID	TC-4.4-02
Title	User switches to Trainer Challenges tab and views trainer-hosted challenges
Pre-conditions	User is logged in. At least one trainer-hosted challenge exists in the DB (trainer field is non-null).
Steps	1. Log in and navigate to the Challenges page. 2. Click the 'Trainer Challenges' tab. 3. Verify the tab becomes active (aria-selected=true) and the displayed list updates. 4. Confirm each trainer challenge card shows 'Hosted by [trainer name]' text. 5. Verify the tab count badge next to 'Trainer Challenges' matches the actual number of trainer challenges shown.
Expected Result	Trainer challenges list renders correctly. Hosted-by label is displayed. Tab count is accurate.
Status	PASS / FAIL

TC-4.4-03: Challenge Progress Updates on Workout Completion

Test ID	TC-4.4-03
Title	Completing a workout increments challenge progress in real time
Pre-conditions	User is logged in. User has already joined an active challenge with goal { workouts: 3 }. User's current_progress.workouts = 0 in DB.
Steps	1. Log in and navigate to the Schedule page. 2. Click 'Start Workout' for today's scheduled session. 3. Click 'Complete Workout'. 4. Verify the success response (toast or modal) shows points awarded. 5. Navigate to any challenge progress view (dashboard or challenges page). 6. Check in the DB (or via the API) that current_progress.workouts is now 1.
Expected Result	WorkoutSession status is 'completed'. UserChallenge.current_progress.workouts incremented by 1. If workouts goal is met, is_completed=True and reward points/badge awarded.

Status	PASS / FAIL
---------------	-------------

TC-4.4-04: Challenge Completion Awards Badge and Points

Test ID	TC-4.4-04
Title	Completing all challenge goals awards the reward badge and bonus points
Pre-conditions	User joined a challenge with goal { workouts: 1 }, reward_points = 100, reward_badge = 'challenge_finisher'. current_progress.workouts = 0.
Steps	1. Log in and navigate to Schedule. 2. Complete a workout session for today. 3. Observe the response: verify points_awarded includes the 100 challenge bonus and newly_unlocked_badges contains 'challenge_finisher'. 4. Go to the Rewards/Profile page and confirm the badge is displayed. 5. Confirm total_points increased by 100 (plus regular session points). 6. In the DB, verify UserChallenge.is_completed = True and completed_at is set.
Expected Result	UserChallenge is marked completed. UserBadge record created. UserPoints total increased. Badge displayed on profile. PointTransaction with reason 'Challenge Completed: ...' is in the DB.
Status	PASS / FAIL

TC-4.4-05: Cannot Join Non-Existent Challenge (Error Path)

Test ID	TC-4.4-05
Title	Joining an invalid or already-joined challenge shows an error
Pre-conditions	User is logged in.
Steps	1. Log in and navigate to Challenges page. 2. Open browser dev tools / network tab. 3. Manually send a POST request to /api/challenges/99999/join/ (non-existent ID). 4. Observe the response status and error message. 5. Alternatively, attempt to join a challenge the user has already joined.
Expected Result	API returns 404 for non-existent challenge, or 400/409 for duplicate join. The UI shows an alert('Failed to join') message. No duplicate UserChallenge is created in the DB.
Status	PASS / FAIL

TC-4.4-06: Empty State When No Challenges Exist

Test ID	TC-4.4-06
Title	Challenges page shows empty message when no challenges are available
Pre-conditions	User is logged in. No active challenges exist in the DB for the selected tab.
Steps	1. Log in and navigate to the Challenges page. 2. If no global challenges exist, verify the message 'No global challenges right now.' is displayed. 3. Switch to Trainer Challenges tab; if none exist, verify 'No trainer challenges right now.' is displayed. 4. Confirm tab count badges show 0.
Expected Result	Empty-state messages are visible. No errors thrown. Tab counts display 0.
Status	PASS / FAIL

Activity 3: Individual Code Review

Code Smell #1: Long Method (`complete_workout_session`)

Description

The `complete_workout_session` view function in `views.py` handles numerous distinct responsibilities: parsing the date, creating/updating the session, inferring duration, awarding points, checking badges, and updating challenge progress for every active challenge. This is a classic Long Method and Single Responsibility Principle (SRP) violation. It is also an example of the Shotgun Surgery smell: any change to challenge logic, points logic, or badge logic requires editing this single massive function.

Location

File: `backend/api/views.py`, function `complete_workout_session`

Code Smell Category

- Long Method (>30 lines of substantive logic per responsibility)
- Single Responsibility Principle violation
- Shotgun Surgery risk

Proposed Fix

Extract challenge progress update logic into a dedicated helper function `_update_challenge_progress(user, session)`. This mirrors the existing `_award_points` and `_check_and_award_badges` helper pattern already used in the same file.

Before (inside complete_workout_session):

--- US 4.4: UPDATE CHALLENGE PROGRESS ---

```
from django.utils import timezone
from .models import UserChallenge
active_user_challenges = UserChallenge.objects.filter(...)
for uc in active_user_challenges:      # ~50 lines of inline challenge logic
```

After (extracted helper): backend/api/views.py

```
#-----
# SMELL-001 FIX: Extracted challenge progress helper
# BUG-002 FIX: Added end_date_gte filter so expired challenges are ignored
#-----

def _update_challenge_progress(user, session, newly_unlocked_badges,
total_points):
    """
    Updates progress for every active, non-expired, non-completed
challenge the user
has joined. Awards points and badges when all goals are met.
Returns the (possibly updated) total_points value.
    """
    today = timezone.localtime()

    active_user_challenges = UserChallenge.objects.filter(
        user=user,
        challenge__is_active=True,
        challenge__end_date__gte=today,    # BUG-002 FIX: skip expired
challenges
        is_completed=False,
    ).select_related('challenge')

    for uc in active_user_challenges:
        criteria = uc.challenge.goal_criteria
        updated = False

        if 'workouts' in criteria:
            uc.current_progress['workouts'] =
uc.current_progress.get('workouts', 0) + 1
            updated = True
```

```

        if 'total_time_minutes' in criteria and session.duration_minutes:
            uc.current_progress['total_time_minutes'] = (
                uc.current_progress.get('total_time_minutes', 0) +
                session.duration_minutes
            )
            updated = True

    if not updated:
        continue

    if all(uc.current_progress.get(k, 0) >= v for k, v in
criteria.items()):
        uc.is_completed = True
        uc.completed_at = timezone.now()
        uc.save()

    if uc.challenge.reward_points > 0:
        user_pts, _ = UserPoints.objects.get_or_create(user=user)
        user_pts.total_points += uc.challenge.reward_points
        user_pts.save()
        total_points = user_pts.total_points

        PointTransaction.objects.create(
            user=user,
            points_awarded=uc.challenge.reward_points,
            reason=f"Challenge Completed: {uc.challenge.name}",
        )

    if uc.challenge.reward_badge:
        badge_obj, created = UserBadge.objects.get_or_create(
            user=user,
            badge_id=uc.challenge.reward_badge,
        )
        if created:
            newly_unlocked_badges.append({
                "badge_id": uc.challenge.reward_badge,
                "name": uc.challenge.reward_badge,
                "description": f"Completed: {uc.challenge.name}",
                "icon": "🏆",
            })

```

```

        "category": "challenge",
        "earned_at": badge_obj.earned_at.isoformat(),
    })

    else:
        uc.save()

    return total_points

```

Code Smell #2: Import Inside Function (Misplaced Imports)

Description

Inside `complete_workout_session`, there are two local import statements: `'from django.utils import timezone'` and `'from .models import UserChallenge'`. Both of these are already imported at the top of `views.py` (`django.utils.timezone` at line 20; `UserChallenge` is imported via the models import block). Importing inside a function is a code smell. While Python caches imports so there is no significant performance penalty, it misleads readers into thinking these are not available at module scope and makes the code harder to maintain.

Location

File: `backend/api/views.py`, inside `complete_workout_session`

Code Smell Category

- Misplaced imports / Dead Code smell
- Indicates rushed or copy-pasted code (Shotgun Surgery artifact)

Proposed Fix

Remove the two duplicate inline imports. Both `django.utils.timezone` and `UserChallenge` are already available at module scope.

Remove these two lines from inside the function:

```

# --- US 4.4: UPDATE CHALLENGE PROGRESS ---
    from django.utils import timezone      # REMOVE: already imported at top
    from .models import UserChallenge      # REMOVE: already imported at top

```

Code Smell #3: Magic Strings / Lack of Constants for Status Values

Description

Throughout `views.py` and `models.py`, session status values are referenced as raw string literals: `'completed'`, `'in_progress'`, `'missed'`. These appear in at least 8 different locations: filter queries, direct assignments, and serializers. This is the Magic Number / Magic String code smell. If the status strings

ever change or a typo is introduced (e.g., 'complete' vs 'completed'), only runtime testing will catch the bug, not the compiler. This also makes search-and-replace refactoring fragile.

Location

File: backend/api/views.py;
multiple locations including workout_history,
complete_workout_session,
auto_mark_missed_sessions,
undo_workout_session.

Code Smell Category

- Magic Strings
- Primitive Obsession

Proposed Fix

Define status constants at the top of views.py (or in a shared constants.py), and replace all raw string literals with these constants:

backend/api/views.py

```
DAYS_OF_WEEK = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday',
'saturday', 'sunday']

# -----
# SMELL-003 FIX: Session status constants – replaces magic strings
throughout
# -----

SESSION_STATUS_IN_PROGRESS = 'in_progress'
SESSION_STATUS_COMPLETED   = 'completed'
SESSION_STATUS_MISSED      = 'missed'
```

Then replace every magic string in the file:

auto_mark_missed_sessions: BEFORE:
status='missed',

AFTER:
status=SESSION_STATUS_MISSED,

start_workout_session: BEFORE:
defaults={"status": "in_progress", "is_completed": False},

```

if session.status != "completed":
    session.status = "in_progress"
# AFTER:
defaults={"status": SESSION_STATUS_IN_PROGRESS, "is_completed": False},
if session.status != SESSION_STATUS_COMPLETED:
    session.status = SESSION_STATUS_IN_PROGRESS

# complete_workout_session; BEFORE:
defaults={"status": "in_progress", "is_completed": False},
session.status = "completed"

# AFTER:
defaults={"status": SESSION_STATUS_IN_PROGRESS, "is_completed": False},
session.status = SESSION_STATUS_COMPLETED

# undo_workout_session; BEFORE:
session.status = "in_progress"

# AFTER:
session.status = SESSION_STATUS_IN_PROGRESS

# workout_history; BEFORE:
qs = WorkoutSession.objects.filter(user=request.user, status_in=['completed', 'missed'])

# AFTER:
qs = WorkoutSession.objects.filter(user=request.user, status_in=[SESSION_STATUS_COMPLETED,
SESSION_STATUS_MISSED])

# _analyze_feedback; BEFORE:
session_status='completed',
# AFTER:
session_status=SESSION_STATUS_COMPLETED,

# get_progress_summary; BEFORE:
status='completed'
# AFTER:
status=SESSION_STATUS_COMPLETED

```

Bug Reports

Bug #1: No UI Validation for Inverted Date Range in History Filter

Field	Detail
Bug ID	BUG-001
Title	History filter accepts end_date before start_date without warning
Severity	Minor
Story	US 3.2: Review Workout History
Steps to Reproduce	1. Go to Workout History page. 2. Set Start Date to 2026-03-01. 3. Set End Date to 2026-01-01. 4. Click Apply Filter.
Actual Result	API is called with invalid range; results may be empty or confusing, no user feedback.
Expected Result	UI should display a validation error: 'End date must be after start date.'
Proposed Fix	Add client-side validation in history/page.tsx before calling <code>fetchHistory()</code>: if <code>endDate</code> and <code>startDate</code> and <code>endDate < startDate</code>, show an error and return early. To summarize: File 1: frontend/src/app/history/page.tsx: 1.1: Add <code>dateError</code> state and validation at the top of <code>fetchHistory</code>. 1.2: Then update the Reset button to also clear the error. 1.3: Add the error message in JSX, right after the closing <code></div></code> of <code>history-filters</code>. File 2: frontend/src/app/history/history.css: <pre> /* BUG-001 FIX: validation error for inverted date range */ .filter-error { color: #ef4444; font-size: 0.85rem; margin-top: 0.25rem; margin-bottom: 0.5rem; font-weight: 500; } </pre>

Bug #2: Challenge Progress Not Filtered by Challenge Date Range

Field	Detail
-------	--------

Bug ID	BUG-002
Title	Expired challenges still receive progress updates on workout completion
Severity	Medium
Story	US 4.4: Participate in Weekly Challenges
Steps to Reproduce	1. Join a challenge. 2. Let the challenge's end_date pass (or set it to yesterday in DB). 3. Complete a workout session.
Actual Result	The _update_challenge_progress loop filters on challenge_is_active=True but does NOT filter on challenge_end_date_gte=today. If is_active is not updated automatically, expired challenges still accumulate progress.
Expected Result	Only challenges whose end_date >= today should receive progress updates.
Proposed Fix	Add challenge_end_date_gte=date.today() to the UserChallenge filter in the challenge progress update block. Fixed is applied inside SMELL-001: the _update_challenge_progress helper includes challenge_end_date_gte=today in the filter.

GitHub Issue References

The following issues were reported on the team's GitHub repository:

- BUG-001: History filter: no validation for inverted date range
- BUG-002: Challenge progress updated for expired challenges
- Code Smell #1: Extract _update_challenge_progress helper from complete_workout_session
- Code Smell #2: Remove duplicate inline imports inside complete_workout_session
- Code Smell #3: Replace magic status strings with named constants

Commit Reference

Proposed refactoring changes (Bugs and Code Smells) have been committed to the team repository. See GitHub commit history for detailed information.